Overflow, printf con la virgola, caratteri di escape

Lezione 5 — 1IOT, SST-ICT-01 — Limiti del char, %f, \n \t \a

22/09/2026

1 Ripasso rapido

Ripasso dei tre tipi di variabili viste finora (`char` a **8 bit**, `int` a 16 bit, `float` con la virgola) e degli **operatori aritmetici**, di **assegnazione** e di **confronto** già introdotti.

2 Il problema dell'overflow: quando il tipo di dato è troppo piccolo

Esempio

Esercizio assegnato per casa: creare due **variabili** `prezzo` e `quantità`, calcolare il loro prodotto in `totale`, stampare il risultato.

```
char prezzo, quantita;
prezzo = 13;
quantita = 22;

char totale;
totale = prezzo * quantita;

printf("%d", totale);
```

Poiché 13 e 22 sono numeri piccoli, il tipo **`char`** sembra sufficiente per contenerli.

Attenzione

Il problema: invece di ottenere 286 (cioè 13×22) come risultato, il programma stampava **30**. La macchina non segnalava nessun **errore**: il programma "funzionava", ma il risultato era sbagliato.

Definizione

Il tipo **`char`** è a **8 bit** e può contenere solo valori da 0 a **255**. Quando un calcolo supera 255, il conteggio **riparte da zero**: questo fenomeno si chiama **overflow**.

Esempio

$286 - 256 = 30$: il valore "gira" oltre il limite massimo del char (256 valori possibili, da 0 a 255) e **riparte da zero**, arrivando a 30. Ecco perché il risultato stampato era 30 invece di 286.

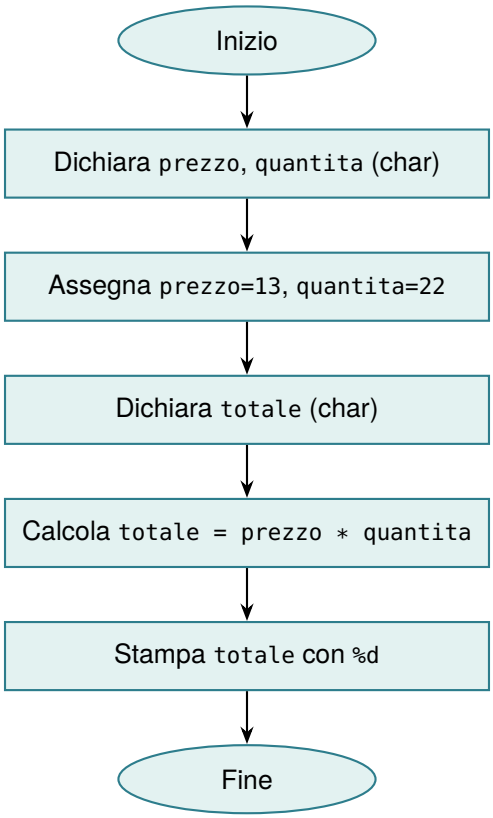


Figura 1: Diagramma di flusso del codice con `char totale`: sequenza lineare di istruzioni, nessuna scelta — ma il risultato finale è comunque sbagliato per overflow.

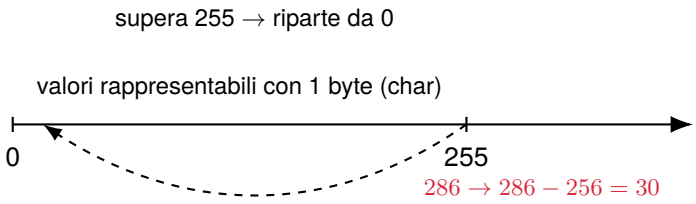


Figura 2: L'overflow del tipo char: un valore che supera 255 non genera un errore, ma "gira" e riparte da zero.

Definizione

Correzione: la **variabile** che riceve il risultato deve essere di un tipo **capiente** abbastanza. Qui basta dichiarare `totale` come `int`:

```
char prezzo, quantita;
prezzo = 13;
quantita = 22;

int totale;
totale = prezzo * quantita;

printf("%d", totale);
```

Va bene anche dichiarare `prezzo` e `quantita` come **`int`**: non è necessario per il calcolo, ma rende il codice più uniforme.

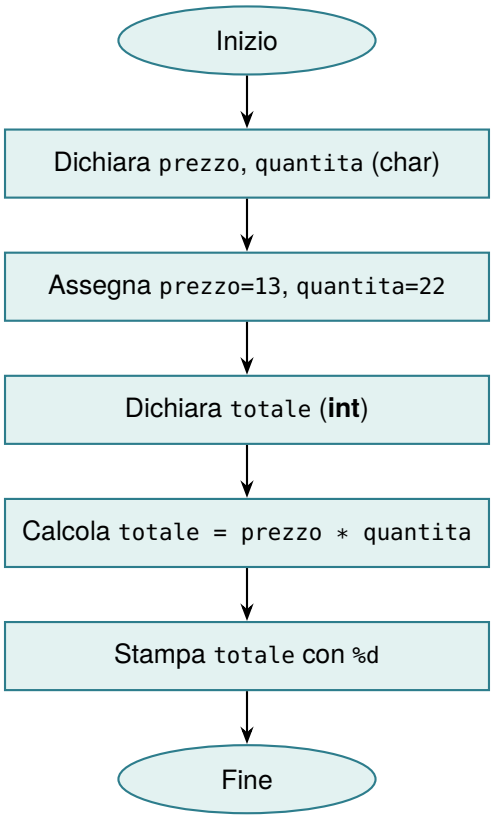


Figura 3: Diagramma di flusso della versione corretta: stessa sequenza di prima, ma `totale` è ora `int` e il calcolo non va più in overflow.

Nota

Chi calcola direttamente `prezzo * quantita` dentro il **`printf`**, senza passare dalla **variabile** `totale`, ottiene comunque il risultato corretto – ma non è una buona pratica: in questo modo il calcolo viene **rieseguito ogni volta** che lo si stampa, consumando inutilmente potenza di calcolo. Meglio calcolare una volta sola in una **variabile** e poi stampare quella.

Attenzione

Morale: fare sempre attenzione al **tipo di dato** e alla dimensione delle **variabili** che si usano. È facile abituarsi a scrivere sempre **`char`** o sempre **`int`** per automatismo, dimenticando di verificare se il risultato di un calcolo può superare la capienza del tipo scelto.

3 Numeri con la virgola: lo specificatore %f

Esempio

Variante dell'esercizio: se il prezzo fosse 13,15 (**con la virgola**), cosa cambia?
Prima osservazione: la **variabile** `quantita`, dichiarata `int`, occupa 4 **byte** anche se basterebbe un tipo più piccolo per un valore come 22 – è uno spreco di **memoria** quando non servono numeri grandi. Il tipo da scegliere dipende sempre dal **contesto**: quantità intere (persone, pezzi) restano **interi**; grandezze che richiedono decimali (es. 1 chilo e 22 grammi di farina)

richiedono il **float**.
La correzione giusta è mettere **float** solo su `prezzo`:

```c
float prezzo;
char quantita;

prezzo = 13.15;
quantita = 22;

float totale;
totale = prezzo * quantita;

printf("%d", totale);
```

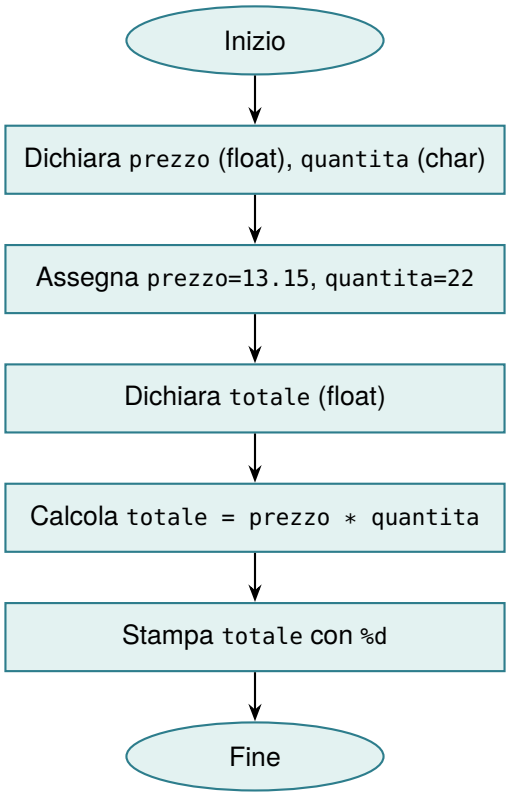


Figura 4: Diagramma di flusso della variante con `prezzo` a virgola: la sequenza è identica, ma `%d` nello stampare un `float` produce un risultato senza senso.

Attenzione

Con **%d** il risultato stampato non ha senso, perché **%d** dice a **printf** "qui arriva un numero **intero**": se gli si passa un valore **con la virgola**, quello che appare a video è un numero senza relazione con il calcolo reale (nell'esempio in aula: veniva stampato `289` invece di `289,3`, perché la parte decimale andava perduta nell'interpretazione).

Definizione

Per stampare correttamente un numero **con la virgola** si usa lo **specificatore** `%f` al posto di `%d`:

```c
printf("%f", totale);
```

3.1 Controllare il numero di cifre decimali

Esempio

Con **%f** da solo, vengono stampate molte cifre dopo la virgola (il formato predefinito ne mostra sei). Per decidere quante cifre mostrare si scrive, tra il `%` e la `f`, la **sequenza** `0.` seguita dal numero di cifre desiderate:

```c
printf("%0.1f", totale);   // una cifra decimale
printf("%0.2f", totale);   // due cifre decimali
printf("%0.3f", totale);   // tre cifre decimali
printf("%0.10f", totale);  // dieci cifre decimali
```

Questa sintassi funziona solo con `%f`: con `%d` non ha senso, perché un numero intero non ha **cifre decimali** da mostrare.

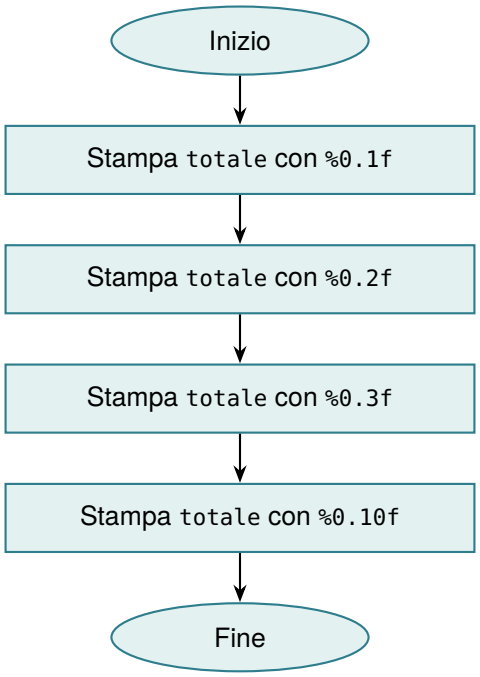


Figura 5: Diagramma di flusso: quattro stampe in sequenza dello stesso valore, ciascuna con un numero diverso di cifre decimali.

Nota

Il valore stampato con `%f` viene **arrotondato** alla cifra decimale **scelta**, non troncato. È un comportamento diverso dalla conversione a numero **intero**, che invece taglia via la parte decimale senza arrotondare.

Esempio

Quante **cifre decimali** usare dipende dal contesto: per il prezzo di un prodotto, due cifre sono di solito sufficienti (i centesimi). Per misure di componenti elettronici, che possono richiedere valori con 6, 7 o 8 cifre dopo la virgola, serve un formato con più cifre (es. `%0.8f` o più).

Nota

Episodio in aula: provando dal vivo l'esempio con $13{,}15 \times 22$ (risultato atteso: $289{,}3$), il programma ha stampato un valore anomalo, con troppe cifre, apparentemente legato al fatto che `quantita` era ancora dichiarata **`char`**. Il docente ha provato a cambiare `quantita` prima in **`int`** poi in **`float`**, senza risolvere l'anomalia, e ha ammesso di non conoscere in quel momento la causa esatta, ipotizzando un comportamento particolare della macchina virtuale online usata per compilare (probabile problema di rappresentazione in virgola mobile, non chiarito in diretta). L'episodio non ha impedito di proseguire con la spiegazione del formato **`%f`**.

4 I caratteri di escape

Definizione

Un **carattere di escape** è una sequenza speciale, che inizia sempre col simbolo **backslash** (\), che non viene stampata come testo letterale ma produce un effetto nella formattazione dell'**output**.

4.1 \n — vai a capo

Esempio

Se dentro una **stringa** tra virgolette compare `\n`, nel punto in cui compare l'output **va a capo**:

```
printf("Il perimetro vale %0.2f\n", perimetro);
printf("L'area vale %0.2f\n", area);
```

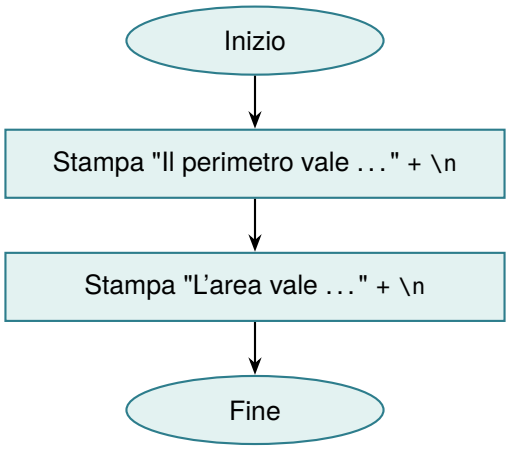


Figura 6: Diagramma di flusso: due stampe in sequenza, ciascuna seguita da un `\n` che manda l'output a capo.

Attenzione

Scrivere il codice C su righe diverse (ad esempio due `printf` distinti, uno per riga) **non** implica automaticamente che l'**output** vada a capo. L'andare a capo nel **codice sorgente** non produce nessun effetto sull'**output**: se manca `\n` dentro la stringa, l'**output** resta sulla stessa riga, anche se il codice che lo genera occupa righe diverse.

Esempio

Dimostrazione fatta in aula: scrivendo `"oggi é una"` e, in un secondo `printf`, `"bella giornata"`, senza `\n` né spazio tra le due stringhe, l'**output** risulta `oggi é unabella giornata` – tutto attaccato, indipendentemente da come il codice é diviso in righe. Aggiungendo uno spazio o `\n` alla fine della prima **stringa**, il risultato torna leggibile.
Si possono inserire più `\n` di seguito: due `\n` consecutivi vanno a capo due volte, cioè saltano una riga vuota in mezzo.

4.2 \t — tabulazione

Definizione

`\t` esegue una **tabulazione**: inserisce un certo numero di spazi (tipicamente configurabile nell'editor, spesso 3 o 4 secondo lo standard usato).

Nota

Perché usare `\t` invece di premere più volte la barra spaziatrice? Perché `\t` occupa **un solo byte** in **memoria**, contro i 4 byte necessari per 4 spazi digitati singolarmente. Su un testo di poche righe la differenza è trascurabile, ma su testi molto lunghi (con moltissimi spazi ripetuti) l'uso della **tabulazione** può ridurre in modo sensibile lo spazio di **memoria** occupato.

4.3 \a — segnale acustico

Definizione

`\a` emette un **segnale acustico** (beep) quando incontrato nell'output.

Esempio

Un uso reale di questo carattere: nei lettori di codici a barre dei supermercati, un beep alla scansione segnala che la lettura è andata a buon fine – se il lettore non emette nessun suono, l'operatore capisce subito che qualcosa non ha funzionato, senza dover guardare lo **schermo**.

5 Esercizio guidato: perimetro e area di un cerchio

Esempio

Traccia (lavagna): dato il raggio r, calcolare **perimetro** e area di un cerchio e stampare i risultati.

$$P = 2 \cdot \pi \cdot r$$
$$A = r \cdot r \cdot \pi$$

con $\pi = 3{,}141592$.

Esempio

Codice scritto insieme in aula:

```
float r;
float PI;

PI = 3.141592;
r = 20;

float perimetro;
float area;

perimetro = 2 * PI * r;
area = r * r * PI;

printf("%0.2f", perimetro);
printf("%0.2f", area);
```

Attenzione

Problema di leggibilità: eseguendo il codice, l'**output** appare come una **sequenza** confusa di cifre e punti (i due risultati stampati uno attaccato all'altro, ad esempio `125.661256.64`), perché tra i due **`printf`** non c'è nessuno spazio né **`\n`**. Aggiungendo uno spazio il risultato diventa leggibile (`125.66 1256.64`), ma resta privo di significato: chi legge non sa quale numero sia il **perimetro** e quale l'area.

Definizione

Soluzione: aggiungere un testo descrittivo prima di ogni **`%f`**, in modo da **contestualizzare** il numero stampato:

```
printf("Il perimetro vale %0.2f\n", perimetro);
printf("L'area vale %0.2f\n", area);
```

Nota

Perché è importante **contestualizzare** i numeri in **output**? Un numero senza etichetta non comunica nulla a chi legge, soprattutto se chi legge l'output non è chi ha scritto il programma.

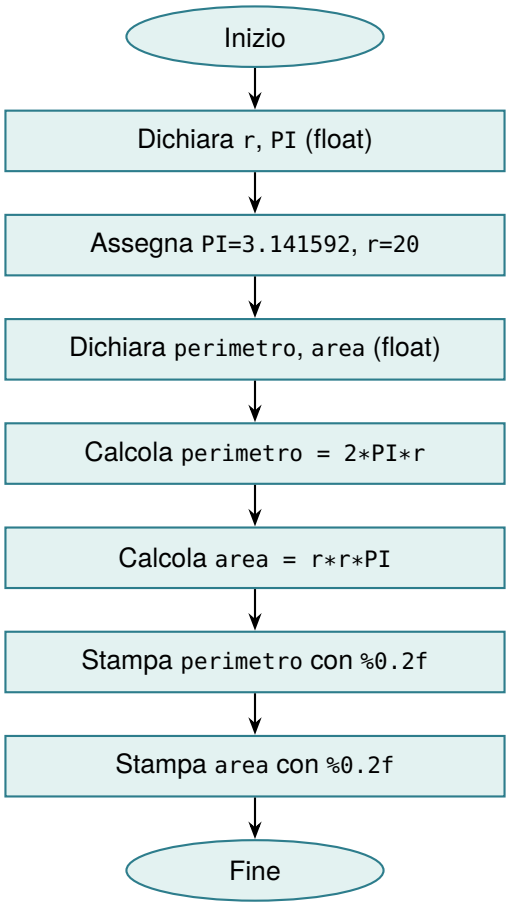


Figura 7: Diagramma di flusso del calcolo di perimetro e area del cerchio: dichiarazioni, assegnazioni, i due calcoli e le due stampe finali, tutti in sequenza.

Se un programma stampa solamente `125,66`, solo chi lo ha scritto sa che si tratta di un **perimetro**: chiunque altro lo utilizzi (ad esempio una persona che usa il programma senza conoscerne il codice) non ha modo di capirlo. Per questo, nei programmi pensati per essere usati da altri, ogni numero stampato dovrebbe essere sempre accompagnato da un testo che ne spieghi il significato.

6 Esercizio: il perimetro di un poligono a 4 lati

Esempio

Traccia (lavagna): dato un poligono a 4 lati a, b, c, d (i cui valori sono già assegnati), scrivere un programma che calcoli il **perimetro** (somma dei lati) e lo stampi.

Esempio

Soluzione, applicando quanto visto sulla contestualizzazione dell'**output**:

```c
float perimetro;
perimetro = a + b + c + d;
printf("Il perimetro vale %0.2f\n", perimetro);
```

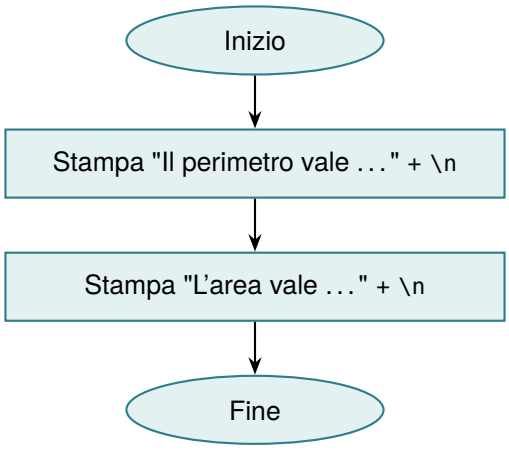


Figura 8: Diagramma di flusso della soluzione contestualizzata: due stampe in sequenza, ciascuna con un'etichetta di testo che spiega il numero.

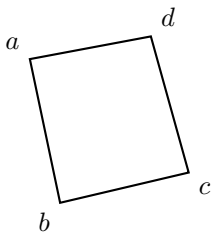


Figura 9: Il quadrilatero disegnato alla lavagna, con i quattro lati etichettati a, b, c, d.

7 Esercizio conclusivo: un solo printf

Esempio

Traccia: scrivere un programma che stampi un **rettangolo pieno** fatto di asterischi, 3 righe da 5 caratteri, usando **un solo** `printf` (non è permesso usarne più di uno):

```
*****
*****
*****
```

Attenzione

Vincolo dell'esercizio: tutto l'**output** deve essere prodotto da un'unica istruzione **`printf`**. Non essendoci numeri da stampare, la soluzione consiste in un'unica **stringa** di testo che combina asterischi e **caratteri di escape** **`\n`** per andare a capo tra una riga e l'altra:

```c
printf("*****\n*****\n*****\n");
```

Esempio

Variante: stampare lo stesso rettangolo ma **vuoto** (solo il bordo fatto di asterischi, il centro vuoto), sempre con un solo **`printf`**:

```
*****
```

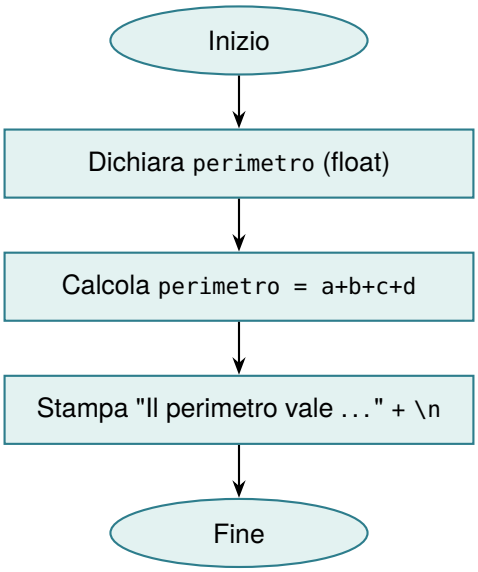


Figura 10: Diagramma di flusso del calcolo del perimetro del quadrilatero: dichiarazione, somma dei quattro lati, stampa contestualizzata.

```
*   *
*****
```

Soluzione: gli spazi nella riga centrale sono caratteri normali come gli asterischi, vanno scritti esplicitamente dentro la **stringa**:

```c
printf("*****\n*   *\n*****\n");
```

Nota

È un esercizio di sintesi su tutto quanto visto in questa lezione: il carattere di escape `\n` per andare a capo, e la costruzione di un **output** complesso (multi-riga) con una singola istruzione `printf`, componendo la **stringa** di formato carattere per carattere.

8 Cosa ricordare di questa lezione

- **Overflow**: un tipo troppo piccolo (es. `char`, max 255) non segnala **errore** ma "gira" e **riparte da zero**, dando un risultato sbagliato senza avviso.
- Scegliere sempre il **tipo di dato** in base al valore massimo che la **variabile** dovrà contenere.
- `%d` è per **interi**, `%f` è per numeri **con la virgola** – non sono intercambiabili.
- `%0.Nf` controlla il numero `N` di **cifre decimali** stampate, arrotondando il valore.
- I **caratteri di escape** iniziano con `\`: `\n` **va a capo**, `\t` tabula, `\a` emette un beep.
- Andare a capo **nel codice sorgente** non equivale ad andare a capo **nell'output**: serve sempre `\n` esplicito.
- I numeri stampati vanno sempre **contestualizzati** con un testo che ne spieghi il significato.

Esempio

Da fare per la prossima lezione: completare l'esercizio del **rettangolo** vuoto di asterischi con un solo `printf`, e verificare anche il calcolo del **perimetro** del poligono a 4 lati con un esempio numerico a **scelta**.